

Spring Boot Integration with Apache Iceberg on AWS Glue

A code construct for reading and writing ML training data stored as Apache Iceberg tables in AWS Glue from a Spring Boot Java application, including auto-creation of the Glue database and Iceberg table.

Below is a comprehensive code construct for integrating Apache Iceberg tables (stored in AWS Glue) with your Spring Boot application for ML training data.

Part 1 — Integration Code

1. Maven Dependencies (pom.xml)

```

<properties>
  <java.version>17</java.version>
  <iceberg.version>1.5.2</iceberg.version>
  <aws.sdk.version>2.25.30</aws.sdk.version>
  <spark.version>3.5.1</spark.version>
</properties>

<dependencies>
  <!-- Spring Boot -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- Apache Iceberg Core -->
  <dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-core</artifactId>
    <version>${iceberg.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-api</artifactId>
    <version>${iceberg.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-data</artifactId>
    <version>${iceberg.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-parquet</artifactId>
    <version>${iceberg.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-aws</artifactId>
    <version>${iceberg.version}</version>
  </dependency>
  <dependency>
    <groupId>org.apache.iceberg</groupId>
    <artifactId>iceberg-aws-bundle</artifactId>
    <version>${iceberg.version}</version>
  </dependency>

  <!-- AWS SDK v2 -->
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>glue</artifactId>
    <version>${aws.sdk.version}</version>
  </dependency>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
    <version>${aws.sdk.version}</version>
  </dependency>
  <dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>sts</artifactId>
    <version>${aws.sdk.version}</version>
  </dependency>

  <!-- Hadoop (required by Iceberg) -->
  <dependency>
    <groupId>org.apache.hadoop</groupId>
    <artifactId>hadoop-common</artifactId>
    <version>3.3.6</version>
    <exclusions>
      <exclusion>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-reload4j</artifactId>
      </exclusion>
    </exclusions>
  </dependency>
</dependencies>

```

2. Application Configuration (application.yml)

```

spring:
  application:
    name: iceberg-ml-data-service

aws:
  region: us-east-1
  glue:
    catalog-name: ml_training_catalog
    database: patents_db
    table: patent_training_data
  s3:
    warehouse-path: s3://your-ml-bucket/iceberg-warehouse/

iceberg:
  catalog:
    impl: org.apache.iceberg.aws.glue.GlueCatalog
    io-impl: org.apache.iceberg.aws.s3.S3FileIO
  bootstrap:
    auto-create: true # set to false in production after first deploy

ml:
  batch-size: 1000
  partition-columns:
    - filing_date
    - tech_center

```

3. Iceberg Catalog Configuration

```

package com.example.mldata.config;

import org.apache.iceberg.CatalogProperties;
import org.apache.iceberg.aws.AwsProperties;
import org.apache.iceberg.aws.glue.GlueCatalog;
import org.apache.iceberg.catalog.Catalog;
import org.apache.hadoop.conf.Configuration;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration as SpringConfig;

import java.util.HashMap;
import java.util.Map;

@SpringConfig
public class IcebergCatalogConfig {

    @Value("${aws.region}")
    private String awsRegion;

    @Value("${aws.glue.catalog-name}")
    private String catalogName;

    @Value("${aws.s3.warehouse-path}")
    private String warehousePath;

    @Bean
    public Catalog icebergCatalog() {
        GlueCatalog catalog = new GlueCatalog();

        Map<String, String> properties = new HashMap<>();
        properties.put(CatalogProperties.CATALOG_IMPL,
            "org.apache.iceberg.aws.glue.GlueCatalog");
        properties.put(CatalogProperties.WAREHOUSE_LOCATION, warehousePath);
        properties.put(CatalogProperties.FILE_IO_IMPL,
            "org.apache.iceberg.aws.s3.S3FileIO");
        properties.put(AwsProperties.CLIENT_REGION, awsRegion);
        properties.put("glue.skip-archive", "true");

        Configuration hadoopConf = new Configuration();
        catalog.setConf(hadoopConf);
        catalog.initialize(catalogName, properties);

        return catalog;
    }
}

```

4. Domain Model

```

package com.example.mldata.model;

import com.fasterxml.jackson.annotation.JsonProperty;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;

import java.time.LocalDate;
import java.util.List;

@Data
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class PatentTrainingRecord {

    @JsonProperty("record_id")
    private String recordId;

    @JsonProperty("application_number")
    private String applicationNumber;

    @JsonProperty("filing_date")
    private LocalDate filingDate;

    private String title;

    @JsonProperty("abstract")
    private String abstractText;

    private List<String> claims;

    private List<String> spec;

    @JsonProperty("skill_group")
    private String skillGroup;

    @JsonProperty("sg_id")
    private String sgId;

    @JsonProperty("sg_id_hash")
    private String sgIdHash;

    @JsonProperty("sg_version")
    private String sgVersion;

    @JsonProperty("uspc_class")
    private String uspcClass;

    @JsonProperty("uspc_subclass")
    private String uspcSubclass;

    @JsonProperty("art_unit")
    private String artUnit;

    @JsonProperty("tech_center")
    private Integer techCenter;

    @JsonProperty("cpc_symbols")
    private List<String> cpcSymbols;

    private String archive;
}

```

5. Schema Definition

```

package com.example.mldata.schema;

import org.apache.iceberg.Schema;
import org.apache.iceberg.types.Types;
import org.springframework.stereotype.Component;

import static org.apache.iceberg.types.Types.NestedField.optional;
import static org.apache.iceberg.types.Types.NestedField.required;

@Component
public class PatentTableSchema {

    public static final String TABLE_NAME = "patent_training_data";

    public Schema getSchema() {
        return new Schema(
            required(1, "record_id", Types.StringType.get()),
            optional(2, "application_number", Types.StringType.get()),
            optional(3, "filing_date", Types.DateType.get()),
            optional(4, "title", Types.StringType.get()),
            optional(5, "abstract", Types.StringType.get()),
            optional(6, "claims", Types.ListType.ofOptional(17, Types.StringType.get())),
            optional(7, "spec", Types.ListType.ofOptional(18, Types.StringType.get())),
            optional(8, "skill_group", Types.StringType.get()),
            optional(9, "sg_id", Types.StringType.get()),
            optional(10, "sg_id_hash", Types.StringType.get()),
            optional(11, "sg_version", Types.StringType.get()),
            optional(12, "uspc_class", Types.StringType.get()),
            optional(13, "uspc_subclass", Types.StringType.get()),
            optional(14, "art_unit", Types.StringType.get()),
            optional(15, "tech_center", Types.IntegerType.get()),
            optional(16, "cpc_symbols", Types.ListType.ofOptional(19, Types.StringType.get())),
            optional(20, "archive", Types.StringType.get())
        );
    }
}

```

6. Repository Layer — Read Operations

```

package com.example.mldata.repository;

import com.example.mldata.model.PatentTrainingRecord;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.iceberg.Table;
import org.apache.iceberg.catalog.Catalog;
import org.apache.iceberg.catalog.TableIdentifier;
import org.apache.iceberg.data.IcebergGenerics;
import org.apache.iceberg.data.Record;
import org.apache.iceberg.expressions.Expressions;
import org.apache.iceberg.io.CloseableIterable;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Repository;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Slf4j
@Repository
@RequiredArgsConstructor
public class IcebergPatentRepository {

    private final Catalog icebergCatalog;

    @Value("${aws.glue.database}")
    private String database;

    @Value("${aws.glue.table}")
    private String tableName;

    private Table loadTable() {
        TableIdentifier identifier = TableIdentifier.of(database, tableName);
        return icebergCatalog.loadTable(identifier);
    }

    /**
     * Scan all records (use with caution on large tables).
     */
    public List<PatentTrainingRecord> findAll(int limit) {
        Table table = loadTable();
        List<PatentTrainingRecord> records = new ArrayList<>();

        try (CloseableIterable<Record> result = IcebergGenerics

```

```

        .read(table)
        .build()) {
    int count = 0;
    for (Record record : result) {
        if (count++ >= limit) break;
        records.add(mapToRecord(record));
    }
} catch (Exception e) {
    log.error("Error reading from Iceberg table", e);
    throw new RuntimeException("Failed to read training data", e);
}
return records;
}

/**
 * Filter by tech center – useful for class-stratified training batches.
 */
public List<PatentTrainingRecord> findByTechCenter(Integer techCenter) {
    Table table = loadTable();
    List<PatentTrainingRecord> records = new ArrayList<>();

    try (CloseableIterable<Record> result = IcebergGenerics
        .read(table)
        .where(Expressions.equal("tech_center", techCenter))
        .build()) {
        for (Record record : result) {
            records.add(mapToRecord(record));
        }
    } catch (Exception e) {
        log.error("Error filtering by tech_center={}", techCenter, e);
        throw new RuntimeException("Failed to query training data", e);
    }
    return records;
}

/**
 * Filter by skill group and date range – common ML training slice.
 */
public List<PatentTrainingRecord> findBySkillGroupAndDateRange(
    String skillGroup, LocalDate from, LocalDate to) {
    Table table = loadTable();
    List<PatentTrainingRecord> records = new ArrayList<>();

    try (CloseableIterable<Record> result = IcebergGenerics
        .read(table)
        .where(Expressions.and(
            Expressions.equal("skill_group", skillGroup),
            Expressions.greaterThanOrEqualTo("filing_date", from.toString()),
            Expressions.lessThanOrEqualTo("filing_date", to.toString())
        ))
        .select("record_id", "application_number", "title", "abstract",
            "claims", "skill_group", "sg_id", "tech_center", "cpc_symbols")
        .build()) {
        for (Record record : result) {
            records.add(mapToRecord(record));
        }
    } catch (Exception e) {
        log.error("Error querying training data", e);
        throw new RuntimeException("Failed to query training data", e);
    }
    return records;
}

@SuppressWarnings("unchecked")
private PatentTrainingRecord mapToRecord(Record record) {
    return PatentTrainingRecord.builder()
        .recordId((String) record.getField("record_id"))
        .applicationNumber((String) record.getField("application_number"))
        .filingDate((LocalDate) record.getField("filing_date"))
        .title((String) record.getField("title"))
        .abstractText((String) record.getField("abstract"))
        .claims((List<String>) record.getField("claims"))
        .spec((List<String>) record.getField("spec"))
        .skillGroup((String) record.getField("skill_group"))
        .sgId((String) record.getField("sg_id"))
        .sgIdHash((String) record.getField("sg_id_hash"))
        .sgVersion((String) record.getField("sg_version"))
        .uspcClass((String) record.getField("uspc_class"))
        .uspcSubclass((String) record.getField("uspc_subclass"))
        .artUnit((String) record.getField("art_unit"))
        .techCenter((Integer) record.getField("tech_center"))
        .cpcSymbols((List<String>) record.getField("cpc_symbols"))
        .archive((String) record.getField("archive"))
        .build();
}
}

```

7. Repository Layer — Write Operations

```
package com.example.mldata.repository;

import com.example.mldata.model.PatentTrainingRecord;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.iceberg.*;
import org.apache.iceberg.catalog.Catalog;
import org.apache.iceberg.catalog.TableIdentifier;
import org.apache.iceberg.data.GenericRecord;
import org.apache.iceberg.data.Record;
import org.apache.iceberg.data.parquet.GenericParquetWriter;
import org.apache.iceberg.io.DataWriter;
import org.apache.iceberg.io.OutputFile;
import org.apache.iceberg.parquet.Parquet;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Repository;

import java.io.IOException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.UUID;

@Slf4j
@Repository
@RequiredArgsConstructor
public class IcebergPatentWriter {

    private final Catalog icebergCatalog;

    @Value("${aws.glue.database}")
    private String database;

    @Value("${aws.glue.table}")
    private String tableName;

    public void writeBatch(List<PatentTrainingRecord> records) {
        TableIdentifier identifier = TableIdentifier.of(database, tableName);
        Table table = icebergCatalog.loadTable(identifier);

        String filepath = table.location() + "/data/" + UUID.randomUUID() + ".parquet";
        OutputFile outputFile = table.io().newOutputFile(filepath);

        try (DataWriter<Record> dataWriter = Parquet.writeData(outputFile)
            .schema(table.schema())
            .createWriterFunc(GenericParquetWriter::buildWriter)
            .overwrite()
            .withSpec(PartitionSpec.unpartitioned())
            .build()) {

            for (PatentTrainingRecord record : records) {
                dataWriter.write(toIcebergRecord(record, table.schema()));
            }

            DataFile dataFile = dataWriter.toDataFile();
            table.newAppend().appendFile(dataFile).commit();
            log.info("Committed {} records to Iceberg table {}", records.size(), tableName);

        } catch (IOException e) {
            log.error("Failed to write batch to Iceberg", e);
            throw new RuntimeException("Failed to write training data", e);
        }
    }

    private Record toIcebergRecord(PatentTrainingRecord src, Schema schema) {
        GenericRecord record = GenericRecord.create(schema);
        Map<String, Object> values = new HashMap<>();
        values.put("record_id", src.getRecordId());
        values.put("application_number", src.getApplicationNumber());
        values.put("filing_date", src.getFilingDate());
        values.put("title", src.getTitle());
        values.put("abstract", src.getAbstractText());
        values.put("claims", src.getClaims());
        values.put("spec", src.getSpec());
        values.put("skill_group", src.getSkillGroup());
        values.put("sg_id", src.getSgId());
        values.put("sg_id_hash", src.getSgIdHash());
        values.put("sg_version", src.getSgVersion());
        values.put("uspc_class", src.getUspcClass());
        values.put("uspc_subclass", src.getUspcSubclass());
        values.put("art_unit", src.getArtUnit());
        values.put("tech_center", src.getTechCenter());
        values.put("cpc_symbols", src.getCpcSymbols());
        values.put("archive", src.getArchive());
    }
}
```

```
        return record.copy(values);
    }
}
```

8. Service Layer

```
package com.example.mldata.service;

import com.example.mldata.model.PatentTrainingRecord;
import com.example.mldata.repository.IcebergPatentRepository;
import com.example.mldata.repository.IcebergPatentWriter;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.stereotype.Service;

import java.time.LocalDate;
import java.util.List;

@Slf4j
@Service
@RequiredArgsConstructor
public class TrainingDataService {

    private final IcebergPatentRepository repository;
    private final IcebergPatentWriter writer;

    public List<PatentTrainingRecord> getTrainingDataByTechCenter(Integer techCenter) {
        log.info("Fetching training data for tech_center={}", techCenter);
        return repository.findByTechCenter(techCenter);
    }

    public List<PatentTrainingRecord> getTrainingSlice(
        String skillGroup, LocalDate from, LocalDate to) {
        return repository.findBySkillGroupAndDateRange(skillGroup, from, to);
    }

    public void ingestTrainingData(List<PatentTrainingRecord> records) {
        log.info("Ingesting {} new training records", records.size());
        writer.writeBatch(records);
    }
}
```

9. REST Controller


```

package com.example.mldata.controller;

import com.example.mldata.model.PatentTrainingRecord;
import com.example.mldata.service.TrainingDataService;
import lombok.RequiredArgsConstructor;
import org.springframework.format.annotation.DateTimeFormat;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDate;
import java.util.List;

@RestController
@RequestMapping("/api/v1/training-data")
@RequiredArgsConstructor
public class TrainingDataController {

    private final TrainingDataService service;

    @GetMapping("/tech-center/{techCenter}")
    public ResponseEntity<List<PatentTrainingRecord>> byTechCenter(
        @PathVariable Integer techCenter) {
        return ResponseEntity.ok(service.getTrainingDataByTechCenter(techCenter));
    }

    @GetMapping("/slice")
    public ResponseEntity<List<PatentTrainingRecord>> getSlice(
        @RequestParam String skillGroup,
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate from,
        @RequestParam @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate to) {
        return ResponseEntity.ok(service.getTrainingSlice(skillGroup, from, to));
    }

    @PostMapping("/ingest")
    public ResponseEntity<String> ingest(@RequestBody List<PatentTrainingRecord> records) {
        service.ingestTrainingData(records);
        return ResponseEntity.ok("Ingested " + records.size() + " records");
    }
}

```

Key Points to Note

A few things worth keeping in mind as you integrate this:

IAM permissions — The runtime needs `glue:GetTable`, `glue:GetDatabase`, `glue:UpdateTable`, plus `s3:GetObject` / `s3:PutObject` on the warehouse bucket. Use IRSA on EKS or instance profiles on EC2 rather than static keys.

Partitioning — Your schema has `filing_date` and `tech_center`, both excellent partition candidates. If the table doesn't already partition on these, pushdown filters in `findByTechCenter` and `findBySkillGroupAndDateRange` will still work, but full file pruning only kicks in when partition columns match.

For large-scale ML reads — `IcebergGenerics` is fine for thousands of rows, but for millions, integrate with Spark via `iceberg-spark-runtime` and stream Parquet files directly to your training pipeline (PyTorch/TensorFlow) instead of materializing rows in JVM memory.

Schema evolution — Iceberg handles add/rename/drop column gracefully. If you later add an `embedding` vector column, use `table.updateSchema().addColumn(...).commit()` and existing reads stay compatible.

Part 2 — Auto-Creating the Catalog, Database, and Table

The implementation in Part 1 **assumes the Glue database and Iceberg table already exist**. Specifically:

- `IcebergCatalogConfig` only initializes a client that talks to Glue; it doesn't create the catalog or database.
- `IcebergPatentRepository.loadTable()` calls `catalog.loadTable(...)` which fails with `NoSuchTableException` if the table doesn't exist.
- `IcebergPatentWriter` also calls `loadTable()` — same failure mode.

As-is, you'd need to create the database and table manually (via AWS Console, CLI, or Athena DDL) before running the app. Iceberg's Java API supports both approaches. The Glue **database** typically should be created once via IaC (Terraform/CloudFormation/Console), but the **table** can absolutely be created from Spring Boot on startup if it doesn't exist.

10. Table Bootstrap Component

Add this component to auto-create the Glue database and Iceberg table on application startup if they do not already exist:

```

package com.example.mldata.config;

import com.example.mldata.schema.PatentTableSchema;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.apache.iceberg.PartitionSpec;
import org.apache.iceberg.Schema;
import org.apache.iceberg.catalog.Catalog;
import org.apache.iceberg.catalog.Namespace;
import org.apache.iceberg.catalog.SupportsNamespaces;
import org.apache.iceberg.catalog.TableIdentifier;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.boot.context.event.ApplicationReadyEvent;
import org.springframework.context.event.EventListener;
import org.springframework.stereotype.Component;

import java.util.HashMap;
import java.util.Map;

@Slf4j
@Component
@RequiredArgsConstructor
public class IcebergTableBootstrap {

    private final Catalog icebergCatalog;
    private final PatentTableSchema schemaProvider;

    @Value("${aws.glue.database}")
    private String database;

    @Value("${aws.glue.table}")
    private String tableName;

    @Value("${iceberg.bootstrap.auto-create:true}")
    private boolean autoCreate;

    @EventListener(ApplicationReadyEvent.class)
    public void bootstrapTable() {
        if (!autoCreate) {
            log.info("Iceberg auto-create disabled, skipping bootstrap");
            return;
        }

        // 1. Ensure namespace (Glue database) exists
        if (icebergCatalog instanceof SupportsNamespaces nsCatalog) {
            Namespace ns = Namespace.of(database);
            if (!nsCatalog.namespaceExists(ns)) {
                log.info("Creating Glue database: {}", database);
                nsCatalog.createNamespace(ns);
            }
        }

        // 2. Ensure table exists
        TableIdentifier identifier = TableIdentifier.of(database, tableName);
        if (icebergCatalog.tableExists(identifier)) {
            log.info("Iceberg table {}.{} already exists - skipping create",
                database, tableName);
            return;
        }

        log.info("Creating Iceberg table {}.{}", database, tableName);

        Schema schema = schemaProvider.getSchema();

        // Partition on tech_center and year of filing_date - good for ML slicing
        PartitionSpec spec = PartitionSpec.builderFor(schema)
            .identity("tech_center")
            .year("filing_date")
            .build();

        Map<String, String> tableProperties = new HashMap<>();
        tableProperties.put("write.format.default", "parquet");
        tableProperties.put("write.parquet.compression-codec", "zstd");
        tableProperties.put("write.metadata.delete-after-commit.enabled", "true");
        tableProperties.put("write.metadata.previous-versions-max", "10");
        tableProperties.put("format-version", "2");

        icebergCatalog.createTable(identifier, schema, spec, tableProperties);
        log.info("Successfully created table {}.{}", database, tableName);
    }
}

```

11. Bootstrap Toggle in application.yml

Add this toggle to `application.yml` so you can switch auto-create off in production:

```
iceberg:
  bootstrap:
    auto-create: true    # set to false in production after first deploy
```

Recommended Approach in Production

There's a tradeoff between convenience and operational safety. Here's the split I'd suggest:

Resource	Recommended approach	Why
Glue database	IaC (Terraform / CloudFormation)	Created once, shared across services, needs IAM/Lake Formation grants
S3 warehouse bucket	IaC	Lifecycle policies, encryption, versioning — not app concerns
Iceberg table	App auto-create on first startup, then disable	Schema lives in code; co-located with the model
Schema evolution	Explicit migration code	Adding columns should be deliberate, reviewed in PR
IAM role/permissions	IaC	Security boundary — never let the app manage this

The pattern I'd use: keep `auto-create: true` during development and the first production deploy, then flip it to `false` so production never silently mutates schema. Schema changes after that go through explicit migration code reviewed in a PR — similar to a Flyway/Liquibase migration but for Iceberg.

Required IAM Permissions for Auto-Create

If you enable auto-create, the service's IAM role needs these in addition to the read/write permissions from Part 1:

```
glue:CreateDatabase      (only if creating the database from the app)
glue:CreateTable
glue:GetDatabase
glue:GetTable
glue:UpdateTable
s3:PutObject             (on the warehouse bucket - for metadata files)
s3:GetObject
s3:ListBucket
```

If you're using **Lake Formation** on top of Glue (common in enterprise setups), you also need Lake Formation grants — `CREATE_TABLE` on the database, and `DATA_LOCATION_ACCESS` on the S3 path. Lake Formation permissions are separate from IAM and easy to miss.

Minimal Manual Setup (if you skip auto-create)

If you'd rather create everything manually once, here's the SQL via Athena — the simplest path:

```
CREATE DATABASE IF NOT EXISTS patents_db;

CREATE TABLE patents_db.patent_training_data (
  record_id          string,
  application_number  string,
  filing_date        date,
  title              string,
  abstract            string,
  claims              array<string>,
  spec               array<string>,
  skill_group         string,
  sg_id              string,
  sg_id_hash          string,
  sg_version          string,
  uspc_class          string,
  uspc_subclass       string,
  art_unit            string,
  tech_center         int,
  cpc_symbols         array<string>,
  archive             string
)
PARTITIONED BY (tech_center, year(filing_date))
LOCATION 's3://your-ml-bucket/iceberg-warehouse/patents_db/patent_training_data/'
TBLPROPERTIES (
  'table_type' = 'ICEBERG',
  'format' = 'parquet',
  'write_compression' = 'zstd'
);
```